Reviewer Pack — Minimal Frobenius Quotient and Communication Complexity

Entry 0c2cb1ac16b9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 13:02:01 UTC

Minimal Frobenius Quotient and Communication Complexity

Entry ID: `0c2cb1ac16b9` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 13:02:01 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Frobenius Quotients) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every k-clique instance, the minimal Frobenius quotient of its associated matrix representation is linearly related to the rank of the communication complexity matrix, such that log_2(min_{A,B} |Fro(A,B)|) = Θ(log(k)) for any 0-1 matrices A and B representing the clique.

Rationale (proposer's reasoning):

> The minimal Frobenius quotient provides an algebraic invariant that could potentially capture the essence of communication complexity through a geometric lens, revealing hidden structures in the problem. This connection has not been explored in complexity theory before, and it offers a novel perspective on understanding communication complexity.

Taxonomy category: `Frobenius_Quotient_Communication_Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d998ab0468eb6259`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For each k-clique instance with n ≤ 40 variables, if the correlation coefficient between the logarithm of the minimal Frobenius quotient and the logarithm of the communication complexity rank across 30 random seeds is greater than 0.5, supporting the conjecture.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"minimal Frobenius quotient" AND "communication complexity" AND matrix` - `"field_A: Representation Theory" AND field_B: Communication Complexity" AND "Frobenius quotients" AND matrix rank` - `"arithmetic properties of communication complexity" AND "representation theory of Frobenius quotients"`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_clique(n, k):
        if n < k:
            return None
        vertices = list(range(n))
        edges = []
        for i in range(k):
            for j in range(i + 1, k):
                edges.append((vertices[i], vertices[j]))
        return vertices, edges

    def matrix_representation(vertices, edges):
        n = len(vertices)
        A = [[0] * n for _ in range(n)]
        B = [[0] * n for _ in range(n)]
        for u, v in edges:
            A[u][v] = 1
            A[v][u] = 1
            B[u][v] = 1
            B[v][u] = 1
        return A, B

    def frobenius_quotient(A, B):
        n = len(A)
        numerator = sum(sum((A[i][j] - B[i][j]) ** 2 for j in range(n)) for i in range(n))
        denominator = sum(sum(A[i][j] ** 2 for j in range(n)) for i in range(n))
        if denominator == 0:
```

```
        return None
    return numerator / denominator

def communication_complexity_rank(A, B):
    n = len(A)
    rank_A = gaussian_elimination(A)
    rank_B = gaussian_elimination(B)
    return min(rank_A, rank_B)

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        if matrix[i][i] == 0:
            for j in range(i + 1, n):
                if matrix[j][i] != 0:
                    matrix[i], matrix[j] = matrix[j], matrix[i]
                    break
            else:
                return None
        pivot = matrix[i][i]
        for j in range(n):
            matrix[i][j] /= pivot
        for j in range(n):
            if j == i:
                continue
            factor = matrix[j][i]
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]
    return sum(1 for row in matrix if any(row))

def log2(x):
    if x <= 0:
        return None
    return math.log2(x)

n = random.randint(5, 40)
k = min(n, random.randint(3, n // 2))
vertices, edges = generate_k_clique(n, k)
if vertices is None or edges is None:
    return {
        "metric_name": "Frobenius Quotient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "k_clique_generation_failed"
    }

A, B = matrix_representation(vertices, edges)
frob_quotient = frobenius_quotient(A, B)
comm_complexity_rank = communication_complexity_rank(A, B)

if frob_quotient is None or comm_complexity_rank is None:
    return {
        "metric_name": "Frobenius Quotient",
        "metric_value": None,
```

```
            "instances_tested": 0,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "matrix_representation_failed"
        }

    if frob_quotient == 0:
        return {
            "metric_name": "Frobenius Quotient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "frob_quotient_zero"
        }

    log_frob_quotient = log2(frob_quotient)
    if log_frob_quotient is None:
        return {
            "metric_name": "Frobenius Quotient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "log2_frob_quotient_failed"
        }

    return {
        "metric_name": "Frobenius Quotient",
        "metric_value": log_frob_quotient,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84a63d5c.py", line 144, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84a63d5c.py", line 97, in run_trial
    comm_complexity_rank = communication_complexity_rank(A, B)
                           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84a63d5c.py", line 54, in communication
    return min(rank_A, rank_B)
           ^^^^^^^^^^^^^^^^^^^
TypeError: '<' not supported between instances of 'NoneType' and 'NoneType'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that the correlation coefficient could not be calculated to assess the conjecture. | next: Investigate and fix the error in the test code to allow for the computation of the correlation coefficient and further validate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14281
2	preregistration	ollama_remote	glm4:latest	0	0	9011
3	novelty	ollama_remote	glm4:latest	0	0	8560
4	novelty	ollama_remote	glm4:latest	0	0	8673
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20555
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16010
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16051
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16170
9	judge	ollama_remote	glm4:latest	0	0	15275

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 124586 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0c2cb1ac16b9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0c2cb1ac16b9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0c2cb1ac16b9.tar.gz` (if generated)